Software Requirements Specification Document

Game Match Making

v1.0
1/22/2026

The Match Makers
Tryston Miller, Phillip Nguyen, Ram Chandra Parmatmuni, Srida Kalidindi, Jeffrey Penn

Submitted in partial fulfillment of the requirements of
IT 326 – Principles of Software Engineering

Table of Contents

TABLE OF CONTENTS **2**

1. INTRODUCTION **4**

1.1 PURPOSE 4
1.2 SCOPE 4
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS 4
1.4 REFERENCES 4
1.5 OVERVIEW 4

2. GENERAL DESCRIPTION **5**

2.1 PRODUCT PERSPECTIVE 5
2.2 USER CHARACTERISTICS 5
2.3 SYSTEM ENVIRONMENT 5
2.4 ASSUMPTIONS AND DEPENDENCIES 5

3. SPECIFIC REQUIREMENTS **6**

3.1 FUNCTIONAL REQUIREMENTS 6
3.1.1 Authenticate Steam OAuth Login *6*
3.1.2 Import Steam Game Library *7*
3.1.3 Request Filtered Games List *7*
3.1.4 Create Group *8*
3.1.5 Delete Group *8*
3.1.6 Transfer Group Ownership *8*
3.1.7 Generate Group Invite Link *9*
3.1.8 Retrieve Group Members *9*
3.1.9 Join Group *9*
3.1.10 Leave Group *9*
3.1.11 Remove Group Member *10*
3.1.12 Vote on Game Selection *10*
3.1.13 Resynchronize Game Library *10*
3.1.14 Resynchronize Installed Games *11*
3.1.15 Retrieve another user's profile *11*
3.1.16 Retrieve Group Session History *11*
3.1.17 Accept Group Join Request *12*
3.1.18 Decline Group Join Request *12*
3.1.19 Retrieve Pending Group Join Requests *12*
3.1.20 Set a playtime requirement for Group Games *13*
3.1.21 Search For a User *13*
3.1.22 Compare Achievements in a Group *13*
3.1.22 Delete user data *14*
3.1.23 Randomize games list *14*
3.1.24 Tag Favorite Game Genres *14*
3.1.25 Retrieve Mutual Friends *14*
3.1.26 Share Weekly Activity Summaries *15*
3.1.27 Share Monthly Activity Summaries *15*
3.1.28 Add Group Preference *15*
3.1.29 Share the final games list *16*
3.1.30 Schedule group game session *16*
3.1.31 Retrieve group calendar *16*
3.1.32 Generate User Profile Report *16*
3.1.33 Create user profile *17*
3.2 NON-FUNCTIONAL REQUIREMENTS 17
3.2.1 Performance *17*
3.2.2 Reliability *17*

3.2.3 Security *17*
3.2.4 Portability *18*

4. DESIGN & DEVELOPMENT 18

1.2 SOFTWARE DEVELOPMENT PROCESS 18
2.2 DELIVERABLE 1 18
4.2.1 Description *18*
a.1.2 Design *19*
a.1.1.2 Class Diagram *19*
a.1.2.2 Activity Diagram *19*
4.2.3 Development *20*

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to describe both the technical and the nontechnical requirements for our web-based Steam Community application. The intended audience includes Professor Rishi Saripalle, and our team of designers. This document will serve as our reference throughout the total development lifecycle and will ensure clear understanding of the application to be developed.

1.2 Scope

This product is an application or service that allows Steam users to connect with one another in the form of smaller friend nodes/communities and being able to compare in game statistics and use them in the form of independent matchmaking and compare game libraries to assist them in finding a game to play. To implement this, the application will let users import their game libraries. The imported libraries can be used to create a list of games algorithmically that the players can collectively enjoy.

What will this product **not** do:

- Replace the Steam storefront or client
- Handle game purchases, downloads, launches, or exists
- Support any forms of chat besides basic text communication

This product will not replace the Steam storefront or client. Rather, the application provides Steam users with a platform to converse with fellow users and identify potential games to enjoy based on their gaming history. The product will not offer

1.3 Definitions, Acronyms, and Abbreviations

SRS: Software Requirements Specification
Steam API: Application Programming Interface provided by Steam for accessing user and game data
OAuth: Open standard for access delegation used for secure Steam account linking

1.4 References

IEEE Std 830-1998: IEEE Guide to Software Requirements Specifications
Steam Web API Documentation: Valve Corporation
OAuth 2.0 Authorization Framework: IETF RFC 6749

1.5 Overview

The rest of this SRS will describe the implementation, design, and audience of the product.

2. General Description

2.1 Product Perspective

This product would be completely independent from any other application. However, application integration should be possible, because it would be an extension of the purpose of the project. In addition, Game Match Making is a standalone web-based application that integrates with the existing Steam ecosystem via its public API. It is not a competitor to Steam but instead a complementary tool meant to build social connections. Unlike Steam's built-in community features, this app is meant to filter and matchmaking based on playtime and achievements. The goal is to offer a more tailored experience to gamers. In addition, the application's primary purpose is to allow groups to better decide what game to play and with a level of ease.

2.2 User Characteristics

The primary users considered PC gamers who use Steam regularly and want a more structured way to determine what game to play with a group of friends.

2.3 System Environment

The application will be run and deployed as a web app accessible via modern browsers such as Chrome, Firefox, Edge, Safari, etc. The backend will be written in Java and use a cloud-based server with a relational database in MySQL. Integration with Steam will occur utilizing the Steam Web API, using OAuth 2.0 for authentication.

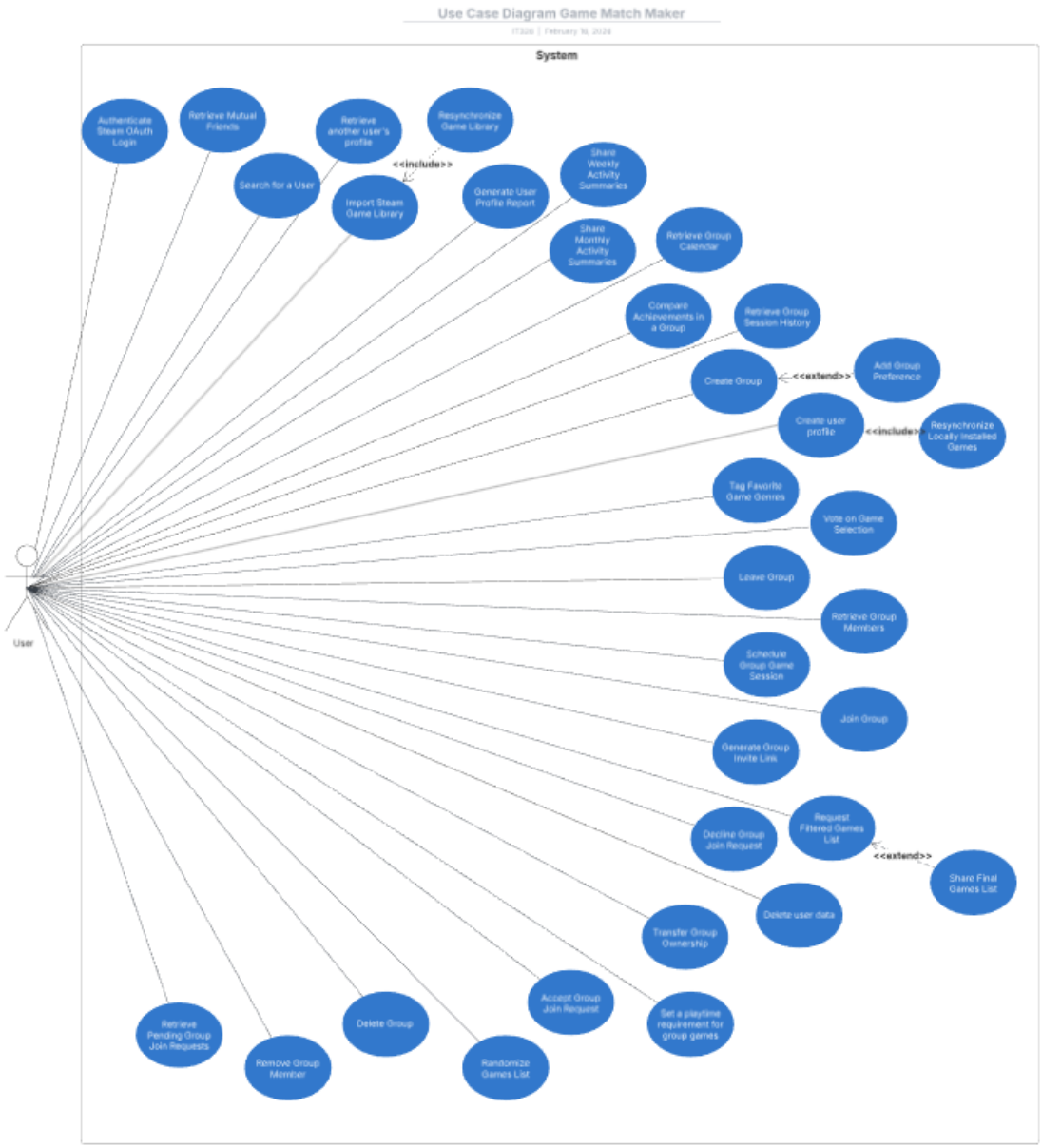
2.4 Assumptions and Dependencies

1. Steam Web API will remain publicly available and unchanged during the development lifecycle. The main reason is that the Steam Web API is very important to our application's functionality, and any changes to the API would change in implementation for the app.
2. Users will maintain a stable internet connection to allow for a connection with the service.
3. Users agree to share public Steam data via the app, because our application requires the user's game usage data which would require some level of permission from users.
4. The app will depend on third-party services for hosting and possible real-time chat, to avoid unnecessary cost to self-hosting.
5. Users will be familiar with Steam and its basic features because the application requires the usage of a Steam account for the user's profile.
6. Users will possess basic web proficiency and navigation skills, due to needing to be able to navigate and use the app.
7. By virtue of being gamers, the users will be familiar with the use of Steam and web applications.

3. Specific Requirements

3.1 Functional Requirements

This section describes specific features of the software project.



3.1.1 Authenticate Steam OAuth Login

- Description – The application shall allow a user to authenticate and create an application account by delegating authentication to their existing Steam account via the Steam OpenID/OAuth 2.0 protocol. No separate username/password for this application is required.
- Actor(s) – User
- Trigger – The user wants to authenticate their user profile with Steam.
- Conditions

- Pre - The user must have a valid Steam account. The user's web browser must be able to connect to both the application's server and Steam's authentication servers.
- Post – The user is successfully authenticated. A user profile is created or retrieved in the application database. The user's basic public Steam profile data (SteamID, persona name, avatar) is imported. The user session is established, and they are redirected to the application dashboard.

3.1.2 Import Steam Game Library

- Description – The application shall allow a user to authenticate and create an application account by delegating authentication to their existing Steam account via the Steam OpenID/OAuth 2.0 protocol. No separate username/password for this application is required.
- Actor(s) – User
- Trigger – The user wants to import their game library.
- Conditions
  - Pre - The user must be successfully authenticated. The user's Steam profile privacy settings must allow their game details to be publicly visible. The Steam Web API must be accessible.
  - Post – The user's owned games list is stored in the application database and associated with their profile. This data is available for use in group matchmaking and profile views.

3.1.3 Request Filtered Games List

- Description – The application allows users to request an updated list of filtered and sorted games for a group. The application allows users to request a list of games filtered by
- Actor(s) – User
- Trigger – A group member requests a filtered and sorted list of games.
- Conditions
  - Pre – User is authenticated, the group exists, and user is a member of the group.
  - Post – A filtered list of games will be created and displayed or sent to the user and their group members in a sorted order.

3.1.4 Create Group

- Description – The application allows users to create a group.
- Actor(s) – User
- Trigger – User requests to create a group.
- Conditions
    - Pre – User is authenticated.
    - Post – Group is created, and the user is set to group owner and sent a notification.

3.1.5 Delete Group

- Description – The application allows a group owner to delete the group.
- Actor(s) – User
- Trigger – User requests to delete group.
- Conditions
    - Pre – User is authenticated, the group exists, and they are the owner of the group in question.
    - Post – Group is deleted, and all members are notified.

3.1.6 Transfer Group Ownership

- Description – The application allows group owners to transfer ownership to another member.
- Actor(s) – User
- Trigger – User requests to transfer ownership to another user.
- Conditions
    - Pre – User is authenticated, the group owner, and the desired new group owner is a member of the group.
    - Post – Ownership is transferred to the designated user and both parties are notified.

3.1.7 Generate Group Invite Link

- Description – The application allows group owners to generate a group invite link for the indicated group.

- Actor(s) – User
- Trigger – User requests a group invite link.
- Conditions
    - Pre – User is authenticated and the group owner.
    - Post – An invite link is generated and sent to the group owner.

3.1.8 Retrieve Group Members

- Description – The application allows authorized users to retrieve a list of members for an indicated group.
- Actor(s) – User
- Trigger – User requests a list of group members.
- Conditions
    - Pre – User is authenticated and if the group is private, they must be a member of the group.
    - Post – List of group members is sent to the user.

3.1.9 Join Group

- Description – The application allows users to join an existing group.
- Actor(s) – User
- Trigger – User requests to join a group.
- Conditions
    - Pre – User is not an existing member of the group, they must be using a group invite link, and the group owner must accept their request.
    - Post – User is added to the group and becomes a group member.

3.1.10 Leave Group

- Description – The application allows users to leave a group of which they are a member.
- Actor(s) – User
- Trigger – User requests to leave a group.
- Conditions
    - Pre – User is authenticated and already a member of the group.

- Post – User is removed from the group and notified.

3.1.11 Remove Group Member

- Description – The application allows users with appropriate permissions to remove a specified member from an existing group.
- Actor(s) – User
- Trigger – User with group owner permissions requests that a user be removed from the group.
- Conditions
  - Pre – User is authenticated, the owner of the group, and the member they're trying to remove is an existing member of the group.
  - Post – Specified user is removed from the group and both parties are notified.

3.1.12 Vote on Game Selection

- Description – The application allows authenticated users to vote on a list of matched games in order to determine which game the group should play.
- Actor(s) – User
- Trigger – User wants to vote on a list of games.
- Conditions
  - Pre – User is authenticated, user is a member of a group, group game list is active, at minimum the group consists of two group members.
  - Post – User's vote is recorded, and vote totals are updated and visible to other group members.

3.1.13 Resynchronize Game Library

- Description – The application allows an authenticated user to manually resynchronize their Steam game library to show newly purchased games and updated playtime/achievements.
- Actor(s) – User
- Trigger – The user wants to update their games library
- Conditions

- Pre – User is authenticated, Steam Web API is accessible, and user's steam profile privacy settings are public to show game details.
- Post – User's game library and playtime/achievements are updated in the application database.

3.1.14 Resynchronize Installed Games

- Description – The application allows an authenticated user to manually resynchronize their locally installed games.
- Actor(s) – User
- Trigger – User wants to update their locally installed games.
- Conditions
  - Pre – User is authenticated and has Steam installed.
  - Post – User's library is updated to include locally installed Steam games, or existing games are marked as installed.

3.1.15 Retrieve another user's profile

- Description – The application allows an authenticated user to view another user's profile which would include their Steam name, avatar, shared games, total playtime and achievements for each game.
- Actor(s) – User
- Trigger – The user wants to view someone's profile.
- Conditions
  - Pre – User is authenticated and target user to view has a profile.
  - Post – The targeted user's profile information is displayed to the user.

3.1.16 Retrieve Group Session History

- Description – The application allows a user to view completed past Game Group sessions
- Actor(s) – User
- Trigger – User wants to view past group session history.
- Conditions
  - Pre – User is a group member of the specified group

- Post – Group session history is displayed.

3.1.17 Accept Group Join Request

- Description – The application allows a user with appropriate authority to accept an incoming join request.
- Actor(s) – User
- Trigger – User wants to respond to an incoming join request from another user.
- Conditions
    1. Pre – User is authenticated, and the user holds elevated group authority.
    2. Post – Request is accepted, and the requesting user is notified.

3.1.18 Decline Group Join Request

- Description – The application allows a user with appropriate authority to decline an incoming join request.
- Actor(s) – User
- Trigger – User wants to respond to an incoming join request from another user.
- Conditions
    3. Pre – User is authenticated, and the user holds elevated group authority.
    4. Post – Request is declined, and the requesting user is notified.

3.1.19 Retrieve Pending Group Join Requests

- Description – The application allows users to retrieve incoming group join requests.
- Actor(s) – User
- Trigger – The user wants to see users who want to join their group.
- Conditions
    5. Pre – User is authenticated, and the user is a group owner.
    6. Post – List of pending group join requests are displayed.

3.1.20 Set a playtime requirement for Group Games

- Description – The application allows the user to set a minimum playtime requirement for games to be considered when finalizing a list.

- Actor(s) – User
- Trigger – User wants to filter out games from the list.
- Conditions
    7. Pre – User is authenticated, and user is the owner of the group
    8. Post – Filtering rules are applied to the group that applies to all games in the group's library

3.1.21 Search For a User

- Description – The application allows the user to find another user in the platform using identifiers.
- Actor(s) – User
- Trigger –User wants to locate another user.
- Conditions
    9. Pre – User is authenticated.
    10. Post – User can see the target user's profile information according to privacy settings if the target user exists.

3.1.22 Compare Achievements in a Group

- Description – Allows a user to compare their achievements with other members of a group.
- Actor(s) – User
- Trigger – User wants to see how their progress compares to other members.
- Conditions
    11. Pre – User is a member of the group. Members have shared achievements.
    12. Post – User can view a comparison of achievements with other group members.

3.1.22 Delete user data

- Description – User can delete the profile and associated data
- Actor(s) – User
- Trigger – User wants to delete their profile.
- Conditions

- Pre – User is authenticated, and Steam data has been imported.
- Post – System notifies user that their data collected has been deleted.

3.1.23 Randomize games list

- Description – User can generate a randomized game unweighted from the custom-tailored algorithm
- Actor(s) – User
- Trigger – User wants a random unweighted game option
- Conditions
  - Pre – User is authenticated, user Steam data has been imported
  - Post – User receives a game determined at random

3.1.24 Tag Favorite Game Genres

- Description – Allows a user to select their favorite game genres.
- Actor(s) – User
- Trigger – User wants to indicate their preferred games.
- Conditions
  13. Pre – User is authenticated. Genres exist on the platform.
  14. Post – User's profile is updated with selected genre tags.

3.1.25 Retrieve Mutual Friends

- Description – Allows a user to see which friends on steam that they share with another user.
- Actor(s) – User
- Trigger – User wants to view shared friends.
- Conditions
  15. Pre – Both users have shared friends on the Steam platform. Both users are authenticated. Privacy settings allow visibility.
  16. Post – User can view mutual friends.

3.1.26 Share Weekly Activity Summaries

- Description – Allows a user to share summaries of their gaming activity over a set period of a week.
- Actor(s) – User
- Trigger – User wants to share activity with others.
- Conditions
    17. Pre – User is authenticated. Activity data exists for the specified period.
    18. Post – User can share their activity summary with friends or followers.

3.1.27 Share Monthly Activity Summaries

- Description – Allows a user to share summaries of their gaming activity over a set period of a month.
- Actor(s) – User
- Trigger – User wants to share activity with others.
- Conditions
    19. Pre – User is authenticated. Activity data exists for the specified period.
    20. Post – User can share their activity summary with friends or followers.

3.1.28 Add Group Preference

- Description – The group can delineate a preference that will be voted on by the whole group, and the everyone needs to say yes for it to be applied.
- Actor(s) – User
- Trigger – User wants to add a group preference to a tag.
- Conditions
    - Pre – User is authenticated.
    - Post – A vote is placed on whether that preference should be included.

3.1.29 Share the final games list

- Description – Any member of the group can share the games list generated
- Actor(s) – User
- Trigger – User wants to share the shared games tier list.
- Conditions

- Pre – User is Authenticated and Games List has been created.
- Post – A permalink would be provided to share.

3.1.30 Schedule group game session

- Description – Allows a user to schedule a future game session
- Actor(s) – User
- Trigger – User wants to schedule a game session for their group.
- Conditions
  - Pre – User is a member of a group, and a matched games list exists for the group.
  - Post – A game session gets scheduled, added to the group calendar, and all group members are notified.

3.1.31 Retrieve group calendar

- Description – Allows a user to retrieve a shared group calendar that displays scheduled game sessions
- Actor(s) – User
- Trigger – User wants to view the group calendar's scheduled activities.
- Conditions
  - Pre – User is a member of a group.
  - Post – The group calendar is displayed, showing scheduled sessions, dates, and times.

3.1.32 Generate User Profile Report

- Description – User can generate a report about their gaming history pulled from their Steam data.
- Actor(s) – User
- Trigger – User wants to get a report on their game history.
- Conditions
  - Pre – User is authenticated.
  - Post – User receives their user profile report.

3.1.33 Create user profile

- Description – User can create a profile and from there either authenticate or maintain basic privileges.
- Actor(s) – User
- Trigger – User wants to create an account with the web application.
- Conditions
    - Pre – User can access the web application.
    - Post – User creates a user profile.

3.2 Non-Functional Requirements

3.2.1 Performance

The system should provide a responsive user experience. Key operations include loading the user's dashboard, importing the Steam game library, and filtering group game lists. These operations should be responsive enough to complete in a few seconds while under typical loading conditions. The system must be able to handle multiple concurrent user sessions without any significant degradation in response times. This is a key requirement in performance as the goal of the application is to handle multiple concurrent users for the groups created. We will improve performance by adhering to programming language best practices and conducting comprehensive unit testing to ensure code efficiency, reliability, and early detection of defects.

3.2.2 Reliability

The application must be dependable for regular user use. Core services such as user authentication, group management, and game library data must maintain a high availability. The system should be able to handle errors with tact including but not limited to temporary Steam API unavailability. The system should prioritize preserving user session data, when possible, to avoid significant application downtime. This could be done by following Exception Handling standards and practices as provided by the Steamworks web API. Debugs are safely output to a logs folder. Included in the backend implementation will be a basic profile that does not require Steam authentication.

3.2.3 Security

User security and data privacy are essential. All user authentication is to be securely delegated to OAuth and its Steam implementation. The application must not store Steam credentials. Data transmission must be encrypted using HTTPS. User data imported from Steam must be accessible only to the user and shared only with explicit consent.

3.2.4 Portability

The application is a web-based service and must be accessible via common desktop web browsers. The backend should be designed for deployment on standard cloud/hosting platforms to avoid dependency on any specific server hardware. To that end we are not building and hosting our own server. In addition, we are developing the primary codebase in Java. This would

enhance portability among multiple OS's. We can use Docker software to ensure that differences in user end systems can be rendered inconsequential to the loading of the web application.

4. Design & Development

The purpose of this section is to outline how the team plans to execute the design and development of the software product(s). It will cover the process model that the team chose to follow and justify it. It will also show a diagram of the process model that will provide details to our processes such as our sprint schedule and meeting frequency.

1.2 Software Development Process

The process model we've decided on is the Scrum model given the structured aspect of deliverables and sprints. The development lifecycle of our application matches best with the deliverable dates that we must adhere to for the submissions. This lets us structure sprints around the submission deadlines and allows for a cycle of planning, design, implementation, testing, and thorough test cases. We plan to have weekly standups on Tuesdays and Thursdays from 6:15 pm to 8:15 pm.

4.2 Deliverable 1

4.2.1 Description

This section explains the team efforts, discussion, and work done for the "Deliverable 1". For example,

> *For this deliverable, we finalized our process design and started to build foundation for our software application. Some of the requirements we started to work on are the ability to login and logout of the application, and the ability of the application to be able to read in a text file.*

21. Design

22. Class Diagram

This section explains the class diagram in detail. The explanation includes the responsibility of each class/abstraction and associations between them. For example,

> *The class diagram showed in Figure 1 captures the vocabulary of the system in terms of Equipment – captures all the equipment leased and stored in the warehouse, Transaction – captures the transaction information related to an equipment, Aisle – captures the location of an equipment.*

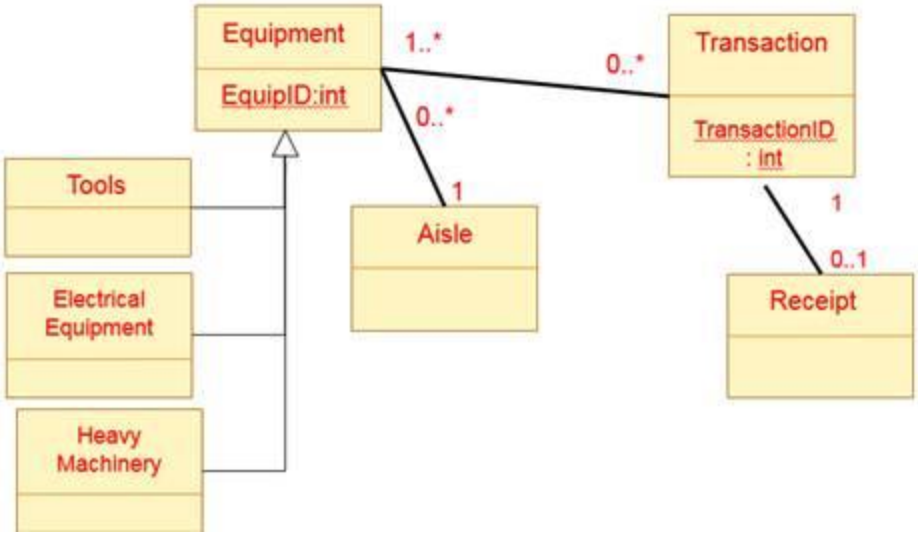


Figure 1: Equipment Class Diagram

4.2.2.2 Activity Diagram

This section explains each activity diagram in detail. The explanation should include the requirement it is modeling. For example,

The activity diagram shown in Figure 2 explains the activities involved and their flow for ordering an equipment (Use case 2: Order Equipment).

The activity diagram shown in Figure

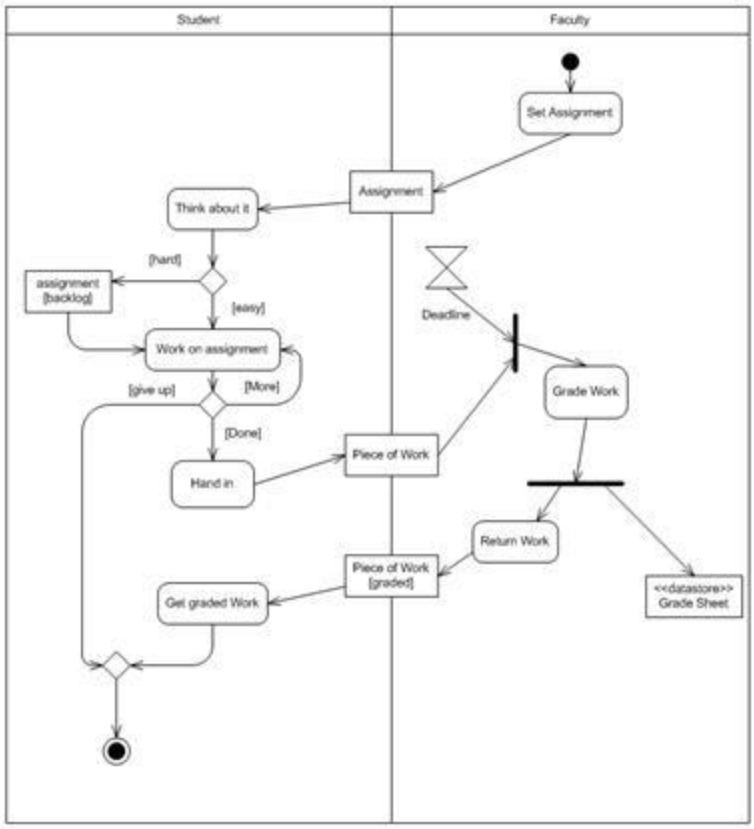


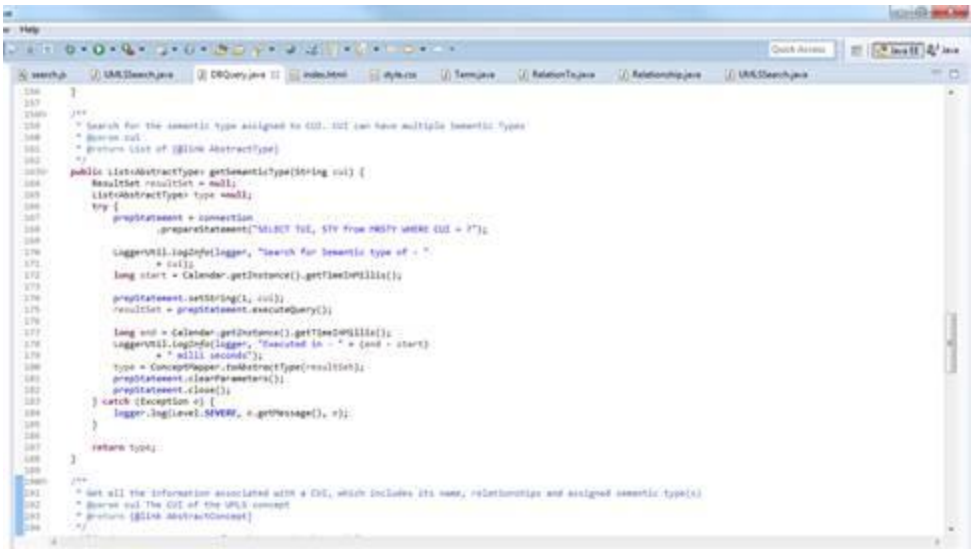
Figure 2: Equipment checkout Activity Diagram

4.2.3 Development

4.2.3.1 Description

*The team MUST have started implementation of the class model or **at least finalized** the development environment (databases, Git, etc.). The team **MUST** provide me their git repository link with all team members on the repo.*

This section describes the development efforts for the deliverable, if any. The team can show their progress on their development by include screenshots of their application or code.



```
    }

    /**
     * Search for the semantic type assigned to CUI. CUI can have multiple Semantic Types
     * @param cui
     * @return List of {@link AbstractType}
     */
    public List<AbstractType> getSemanticType(String cui) {
        ResultSet resultSet = null;
        List<AbstractType> type = null;
        try {
            prepStatement = connection
                    .prepareStatement("SELECT TUI, STY from MRSTY WHERE CUI = ?");

            LoggerUtil.logInfo(logger, "Search for Semantic type of - "
                    + cui);
            long start = Calendar.getInstance().getTimeInMillis();

            prepStatement.setString(1, cui);
            resultSet = prepStatement.executeQuery();

            long end = Calendar.getInstance().getTimeInMillis();
            LoggerUtil.logInfo(logger, "Executed in - " + (end - start)
                    + " milli seconds");
            type = ConceptMapper.toAbstractType(resultSet);
            prepStatement.clearParameters();
            prepStatement.close();
        } catch (Exception e) {
            logger.log(Level.SEVERE, e.getMessage(), e);
        }

        return type;
    }

    /**
     * Get all the information associated with a CUI, which includes its name, relationships and assigned semantic type(s)
     * @param cui The CUI of the UMLS concept
     * @return {@link AbstractConcept}
     */
```